Rapport — Virtualisation et conteneurisation : Annalis

Plateforme de Partage de Documents Académiques

Thomas FRIGANT & Philippe AUMAITRE
Polytech Marseille

No Institute Given

1 Conception de l'application

Ce projet est une application web destinée aux étudiants afin de faciliter le partage et la gestion de documents académiques. Lorsqu'un étudiant se connecte à l'application, il peut consulter les documents partagés par ses pairs, en prévisualiser le contenu directement dans le navigateur et télécharger les fichiers qui l'intéressent. Il peut également uploader ses propres documents en leur associant des métadonnées (matière, secteur, année universitaire, année scolaire).

L'architecture de l'application repose sur trois services principaux :

- **Couche client/proxy (Vue 3 / Vite / Nginx)**, qui s'occupe de l'interface utilisateur côté étudiant et sert les fichiers statiques compilés en production.
- **Couche métier (Python / FastAPI)**, elle gère l'authentification OAuth2/JWT, l'upload et la gestion des fichiers, ainsi que les requêtes de filtrage et de recherche sur les documents.
- **Base de données (MariaDB 11)**, qui gère le stockage des utilisateurs, des métadonnées des documents et de l'historique de consultation. Le schéma des différentes tables est géré automatiquement via SQLAlchemy au démarrage du backend.

Pour le prototype de l'application, il a été choisi de concevoir une interface web simple et intuitive pour l'utilisateur. Le prototype a été réalisé sur **Figma**.

Le prototype possède les pages suivantes :

- **Connexion / Inscription** : formulaires d'authentification avec stockage sécurisé du token JWT ;
- **Page principale** : affichage des documents consultés récemment par l'utilisateur connecté ;
- **Explorer** : liste paginée des documents partagés avec filtres / barre de recherche;
- **Upload de document** : formulaire de partage de fichier avec saisie des métadonnées associées ;
- **Prévisualisation** : streaming du document pour affichage direct dans le navigateur sans téléchargement préalable ;

1.1 API REST

L'API est prefixée par `/api`. L'authentification via token JWT est requise pour la plupart des endpoints.

Méthode	Endpoint
POST	/api/auth/register
POST	/api/auth/login
POST	/api/documents/upload
GET	/api/documents/view
GET	/api/documents/{doc_id}/preview
GET	/api/documents/{doc_id}/download
GET	/api/documents/recent

2 Virtualisation

Pour déployer l'application, nous avons utilisé des **machines virtuelles** sur VirtualBox. L'architecture en trois couches est conservée :

Machine	Rôle	Configuration
1	Serveur Frontend (Nginx)	1 cœur CPU, 1 Go de RAM
2	Serveur Backend (Python / FastAPI)	2 cœurs CPU, 2 Go de RAM
3	Base de données MariaDB	2 cœurs CPU, 2 Go de RAM

Chaque machine utilise **Ubuntu Server** comme système d'exploitation. Le choix de ce système d'exploitation a été motivé par la stabilité, la richesse du système de paquets apt et la disponibilité des guides et ressources, qui reposent souvent sur l'utilisation d'Ubuntu. L'entièreté des services nécessaires pour déployer l'application sont disponibles directement avec une seule commande.

Les trois machines virtuelles sont connectées sur un **réseau privé interne** permettant d'isoler les couches métier et base de données des clients. L'accès à l'application se fait à travers l'interface NAT de la machine Nginx. Cependant, pour simplifier la configuration, une interface NAT a été installée sur l'ensemble des machines afin d'y accéder par SSH. Cela permet d'accéder à la machine directement depuis l'invite de commande de l'hôte plutôt que de se connecter par l'écran virtuel de VirtualBox.

La configuration des machines virtuelles est un processus qui demande beaucoup de temps car il faut installer le système d'exploitation sur chaque machine, installer les paquets nécessaires et configurer les services pour que l'application fonctionne correctement. De plus, dupliquer les machines virtuelles pour augmenter les capacités de l'application est un processus long : le clonage du disque dur virtuel n'est pas suffisant car il faudra reconfigurer certains services (adresses IP, noms d'hôtes, certificats) pour permettre la coexistence des différentes machines sur le même réseau.

3 Conteneurisation

3.1 Principe

Une machine virtuelle embarque un système d'exploitation complet avec son propre noyau, ce qui explique le temps d'installation et la quantité de ressources qu'elle réserve. Un conteneur, lui, partage le noyau de la machine hôte et n'embarque que l'application et ses dépendances. Il est donc plus léger, plus rapide à démarrer et ne réserve pas de ressources fixes à l'avance.

Nous avons conservé l'architecture en trois couches du déploiement en machines virtuelles. Chaque machine virtuelle est remplacée par un conteneur, et l'ensemble est décrit dans un unique fichier `docker-compose.yml` géré par Docker Compose.

Couche	Machine virtuelle	Conteneur	Image utilisée
Client / proxy	Machine 1 (Alpine)	`proxy`	construite à partir de `node 22 (build) + nginx`
Métier	Machine 2 (Ubuntu)	`app`	construite à partir de `python:3.12-slim`
Base de données	Machine 3 (Ubuntu)	`db`	`mariadb:11` (officielle)

3.2 Construction des images

La couche base de données directement une image officielle : seule ça configuration est fournie. Les couche métier et client, elles, sont construite à partir des `Dockerfile`, qui reprend les étapes de la configuration manuelle de la machine virtuelle :

Pour le client, à partir d'une image node, les packages sont téléchargés puis le build de l'application est lancé. Ensuite, avec l'image nginx, le build statique est copié vers le répertoire web de nginx ce qui permet d'avoir une image légère sans node.js. Pour le métier, à partir d'une image Python légère, les dépendances listées dans `requirements.txt` sont installées.

3.3 Orchestration avec Docker Compose

Le fichier `docker-compose.yml` remplace à lui seul la configuration réseau et l'installation des trois machines :

- **Réseau** : Docker crée un réseau privé dans lequel les conteneurs se trouvent par leur nom. Le backend se connecte à la base avec DB_HOST=annalis. Nginx reverse-proxy les requêtes API vers http://app:8000. Les adresses IP fixes disparaissent.

- **Isolation** : seul le conteneur frontend publie un port (8080) vers l'extérieur. La base de données et la couche métier ne sont joignables que depuis les autres conteneurs, ce qui reproduit l'isolation obtenue avec le réseau interne de VirtualBox.
- **Persistance** : les données de MariaDB sont stockées dans un volume Docker (db_data). Les fichiers uploadés sont également persistés dans un volume documents. Elles sont conservées lorsque les conteneurs sont supprimés ou recréés.
- **Initialisation de la base** : les scripts de migration SQLAlchemy sont exécutés automatiquement au premier démarrage du backend. La création des tables est faite par l'ORM à partir des modèles définis dans le code.
- **Ordre de démarrage** : le conteneur backend dépend du conteneur db et attend que la base soit réellement prête avant de se lancer, ce qui évite les erreurs de connexion au démarrage.
- **Secrets** : les variables d'environnements sont lus depuis un fichier .env, un fichier .env.exemple est fourni.

3.4 Déploiement

Là où le déploiement par machines virtuelles demandait d'installer trois systèmes, de configurer le réseau et de suivre une longue procédure manuelle, l'application complète se lance désormais avec quelques commandes :

- cp .env.example .env
- docker compose build
- docker compose up -d

L'application est alors accessible sur le port `8080` de la machine hôte. L'arrêt se fait avec `docker compose down` (les données sont conservées) et un retour à l'état initial avec `docker compose down -v`.

4 Tests de performance

Pour tester les performances de l'application en charge, nous avons utilisé **Apache JMeter**. Un script JMeter génère des requêtes HTTP vers l'application pour mesurer le temps de latence, le nombre de transactions par seconde et le taux d'erreur.

Pour mesurer en même temps les métriques d'utilisation du CPU, de la mémoire et du disque dur, nous avons utilisé un script d'accompagnement qui effectue une charge durant 30 secondes. Pendant les 10 premières secondes, le script augmente progressivement le nombre de clients connectés à l'application, puis pendant les 20 secondes restantes effectue les requêtes pour un nombre de clients configurable lors de l'exécution.

Lors des tests de performance, une seule instance de l'application est lancée, avec une base de données pré-remplie.

Pour chaque test effectué, une page HTML présentant les résultats détaillés est disponible dans le dossier `charge/results` du dépôt.

4.1 Résultats – Machines virtuelles

Charge	Débit (tx/s)	Latence Médiane (ms)	Latence 90e (ms)	Latence 95e (ms)	Latence 99e (ms)	Erreur (%)
1	0.00%	147.00	270.00	270.00	270.00	6.21
5	0.00%	132.00	263.50	264.00	264.00	2.40
10	0.00%	124.50	242.20	286.70	289.00	4.31
25	0.00%	239.00	442.60	457.25	466.00	10.16
50	0.00%	2084.50	3301.10	3618.05	4081.26	13.06
100	0.00%	3733.50	11094.60	11334.55	11749.73	13.20
150	0.00%	5363.00	17097.40	17498.70	17953.95	13.62
200	12.00%	6821.00	22781.00	23134.60	23566.50	14.27
300	43.83%	3882.00	23820.00	24022.75	24524.66	21.10

On constate que nos machine virtuelle ne supporte pas la charge dès 200 utilisateurs. Lors du test à 300 utilisateurs, l'application est saturée et ne peut plus traiter les requêtes correctement : le taux d'erreur atteint 43,83%.

4.2 Résultats – Conteneurs

Charge	Débit (tx/s)	Latence Médiane (ms)	Latence 90e (ms)	Latence 95e (ms)	Latence 99e (ms)	Erreur (%)
1	0.00%	115.00	226.80	255.50	257.00	4.31
5	0.00%	116.00	244.90	247.00	247.00	2.41
10	0.00%	115.00	226.80	255.50	257.00	4.31
25	0.00%	117.50	227.90	246.70	259.00	10.15
50	0.00%	655.00	2526.40	2629.55	2929.99	14.65
100	0.00%	3899.00	9416.80	9737.40	10410.15	13.95
150	0.00%	9063.00	17176.00	17373.65	17869.85	13.49
200	0.00%	13007.00	23835.80	24176.70	24566.38	13.40
300	16.00%	17529.50	33388.50	34697.05	35158.81	14.47

On constate que pour les conteneurs, la situation est meilleur. Ils arrivent à tenir la charge de 200 utilisateurs en même tant mais dès 300 utilisateurs, ils commencent à être saturés avec 16% d'erreurs.

4.3 Analyse des résultats

Les tests de performance confirment l'intérêt des conteneurs dans notre cas. La charge supportée est plus élevé pour les conteneurs que pour les machines virtuelles. À 200 utilisateurs, l'application en machines virtuelles n'était capable de répondre pour certaines requête (12% d'erreurs), alors que la version conteneurisée traitait toutes les requêtes.

5 Conclusion

Déployer Annalis dans deux infrastructures différentes nous a permis de mesurer concrètement les forces et les faiblesses de chaque approche. Le déploiement en machines virtuelles, bien qu'offrant une isolation parfaite, s'est révélé lourd et peu reproductible. Chaque duplication nécessitait une reconfiguration manuelle des réseaux, des adresses IP et des services, rendant l'opération fastidieuse et sujette aux erreurs.

La conteneurisation a transformé cette expérience. Un fichier docker-compose.yml unique remplace désormais l'ensemble de la configuration infrastructurelle. Le démarrage de l'application se résume à une commande, et la reproductibilité est garantie par le versionnement des images et des configurations.

De plus, les tests de performances ont montré que pour une application web comme celle-ci, les conteneurs étaient supérieur en terme de traitement des requêtes.

En somme, les machines virtuelles conservent leur utilité pour l'isolation forte, mais les conteneurs représentent l'avenir du déploiement d'applications web : agiles, économiques et évolutives.

Références

[1] Dépôt GitHub Annalis. Disponible sur : https://github.com/FuzeTea83/Annalis
[2] Documentation FastAPI. https://fastapi.tiangolo.com/
[3] Documentation Vue.js. https://vuejs.org/
[4] Documentation Docker. https://docs.docker.com/
[5] Documentation Apache JMeter. https://jmeter.apache.org/
[6] Documentation MariaDB. https://mariadb.com/